Q1/Month

Year

SHEQ4 in C

Company Name

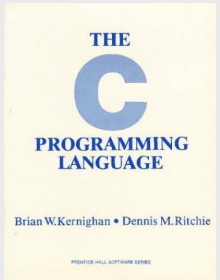
This is your space for legal copy. State whether the information is confidential and specify who can read this deck.

Company Name	Q1 / Month	Year	

Today's agenda

- Language Overview
- Implementation
- C Values, Syntax, and Scoping
- C Types
- Conclusion

Brief C Overview

The History of C	Origins	Evolution	Legacy
	Created 1972 by Dennis Ritchie at Bell Labs	K&R C (1978) - "The C Programming Language" book set the standard	Foundation for C++, influenced Java, C#, JavaScript
	Developed to rewrite UNIX (previously in assembly)	ANSI C (1989) - First official standardization	Still dominant in systems programming and embedded systems
			

Implementation

Tests

- We started with many of the same tests we used in the racket version of SHEQ4
- We developed a simplified a simplified ‘test’ function which calls out serialize and interp
- Though parsing was attempted we mainly focused out work on test abstract syntax

Results

```
krutledg@unix3:~/csc_430/sheq4 $ ./sheq4_simple
{+ 3 4}                         => 7
{- 10 3}                        => 7
{* 6 7}                         => 42
{/ 15 3}                        => 5
{<= 3 5}                        => true
{<= 5 3}                        => false
{if true 1 2}                   => 1
{if false 1 2}                  => 2
{{lam (x y) {+ x y}} 3 4}       => 7
{let [x=3 y=4] {+ x y}}         => 7
```

```c
void test(char *expr, Expr *e, Env *env) {
    printf("%-28s => %s\n", expr, serialize(interp(e, env)));
}

int main(void) {
    Env env = top_env();

    test("{+ 3 4}",   APP(ID("+"), NUM(3), NUM(4)), &env);
    test("{- 10 3}",  APP(ID("-"), NUM(10), NUM(3)), &env);
    test("{* 6 7}",   APP(ID("*"), NUM(6), NUM(7)), &env);
    test("{/ 15 3}",  APP(ID("/"), NUM(15), NUM(3)), &env);
    test("{<= 3 5}",  APP(ID("<="), NUM(3), NUM(5)), &env);
    test("{<= 5 3}",  APP(ID("<="), NUM(5), NUM(3)), &env);

    test("{if true 1 2}",  IF(ID("true"), NUM(1), NUM(2)), &env);
    test("{if false 1 2}", IF(ID("false"), NUM(1), NUM(2)), &env);

    test("{{lam (x y) {+ x y}} 3 4}",
         APP(LAM("x", "y", APP(ID("+"), ID("x"), ID("y"))), NUM(3), NUM(4)), &env);

    // let desugars to immediate lambda application
    test("{let [x=3 y=4] {+ x y}}",
         APP(LAM("x", "y", APP(ID("+"), ID("x"), ID("y"))), NUM(3), NUM(4)), &env);

    return 0;
}
```

Implementation

Data Definitions

- Data definitions were used using C's structs
- Tagged unions were used to build the Expr and Value types
  - Tagged with enum
  - Union describes the actual structures
- For Expr made of other Exprs pointers are used
- For values like params or argos where it wasn't a single struct being referenced a double pointer was used

Compound Literals (Anonymous Structs)

```c
// compound literals: &(Expr){...} allocates on stack, no malloc needed
#define NUM(n)      &(Expr){.type = NumC, .num = n}
#define ID(s)       &(Expr){.type = IdC, .var = s}
#define IF(c,t,e)   &(Expr){.type = IfC, .ifc = {c, t, e}}
#define LAM(x,y,b)  &(Expr){.type = LamC, .lamc = {(char*[]){x, y}, 2, b}}
#define APP(f,a,b)  &(Expr){.type = AppC, .appc = {f, (Expr*[]){a, b}, 2}}

Value numv(double n)   { return (Value){.type = NumV, .num = n}; }
Value boolv(int b)     { return (Value){.type = BoolV, .bool = b}; }
Value primv(char *op)  { return (Value){.type = PrimV, .prim = op}; }
```

```c
// forward declaration needed because Expr contains pointers to itself
typedef struct Expr Expr;
struct Expr {
    enum { NumC, IdC, IfC, LamC, AppC } type;
    union {
        double num;
        char *var;
        struct { Expr *test, *then_e, *else_e; } ifc;
        struct { char **params; int paramc; Expr *body; } lamc;
        struct { Expr *fun; Expr **args; int argc; } appc;
    };
};

typedef struct Env Env;
typedef struct Value Value;

struct Value {
    enum { NumV, BoolV, ClosV, PrimV } type;
    union {
        double num;
        int bool;
        struct { char **params; int paramc; Expr *body; Env *env; } closv;
        char *prim;
    };
};
```

Implementation

Interpreter

- Primarily build on C's switch operator, based on the type of the express
- At the correct term the '→' operator is used to dereference the value of the structure which is needed.
- For IdC an additional lookup helper was developed
- Not significantly more complicated than racket match patterns, just more syntax

```
Value interp(Expr *e, Env *env) {
    switch (e->type) {
    case NumC:
        return numv(e->num);

    case IdC: {
        Value *v = lookup(e->var, env);
        if (!v) { printf("SHEQ: unbound '%s'\n", e->var); exit(1); }
        return *v;
    }

    case IfC: {
        Value c = interp(e->ifc.test, env);
        if (c.type != BoolV) { printf("SHEQ: if needs bool\n"); exit(1); }
        return c.bool ? interp(e->ifc.then_e, env) : interp(e->ifc.else_e, env);
    }

    case LamC:
        return (Value){.type = ClosV, .closv = {
            e->lamc.params, e->lamc.paramc, e->lamc.body, env
        }};
```

Implementation

Function Application

- Because we needed to allocate an array for interpreted arguments ArgC now has a value of argc, which could be set at parse time
- Next the application is handled depending on its type is a similar fashion to our racket code
- Information about out Env and extend functions are on the next slide

Apply Primitive

```
Value apply_primitive(char *op, Value *args) {
    double a = args[0].num, b = args[1].num;
    if (strcmp(op, "+") == 0)  return numv(a + b);
    if (strcmp(op, "-") == 0)  return numv(a - b);
    if (strcmp(op, "*") == 0)  return numv(a * b);
    if (strcmp(op, "/") == 0)  return numv(a / b);
    if (strcmp(op, "<=") == 0) return boolv(a <= b);
    printf("SHEQ: unknown primitive '%s'\n", op); exit(1);
}
```

```
case AppC: {
    Value fn = interp(e->appc.fun, env);
    int argc = e->appc.argc;

    Value args[argc];
    for (int i = 0; i < argc; i++)
        args[i] = interp(e->appc.args[i], env);

    if (fn.type == PrimV)
        return apply_primitive(fn.prim, args);

    if (fn.type == ClosV) {
        Env new_env = extend(fn.closv.env, fn.closv.params, args, argc);
        return interp(fn.closv.body, &new_env);
    }

    printf("SHEQ: not a function\n"); exit(1);
}
}
exit(1);
```

Implementation

Environment

- Because we needed to allocate an array for interpreted arguments ArgC now has a value of argc, which could be set at parse time
- Next the application is handled depending on its type is a similar fashion to our racket code
- Information about out Env and extend functions are on the next slide

Top Env

```
Env top_env(void) {
    Env env = {.size = 7, .parent = NULL};
    env.bindings[0] = (Binding){"+",     primv("+")};
    env.bindings[1] = (Binding){"-",     primv("-")};
    env.bindings[2] = (Binding){"*",     primv("*")};
    env.bindings[3] = (Binding){"/",     primv("/")};
    env.bindings[4] = (Binding){"<=",    primv("<=")};
    env.bindings[5] = (Binding){"true",  boolv(1)};
    env.bindings[6] = (Binding){"false", boolv(0)};
    return env;
}
```

```
struct Env {
    Binding bindings[64];
    int size;
    Env *parent;
};
```

```
Value *lookup(char *name, Env *env) {
    while (env) {
        for (int i = 0; i < env->size; i++)
            if (strcmp(env->bindings[i].name, name) == 0)
                return &env->bindings[i].val;
        env = env->parent;
    }
    return NULL;
}

Env extend(Env *parent, char **params, Value *vals, int n) {
    Env e = {.size = n, .parent = parent};
    for (int i = 0; i < n; i++)
        e.bindings[i] = (Binding){params[i], vals[i]};
    return e;
}
```



What C Has (and Lacks)

- Function pointers exist but cannot capture environments
- ‘A’ is just the number 65
- You manage memory yourself

Feature	Racket	C
Numbers	Yes	Yes (int, double, char)
Strings	Yes	No (char arrays)
Symbols	Yes	No
Closures	Yes	No
Objects	Yes	No (structs only)
Garbage Collection	Yes	No

Statements Don’t Return Values

- `if` is a statement, ternary `?:` is an expression
- Block scope with `{ }`, variable die when block ends
- No nested functions, no classes, no methods

```c
// Racket: (if (> x 0) 1 2) returns either 1 or 2

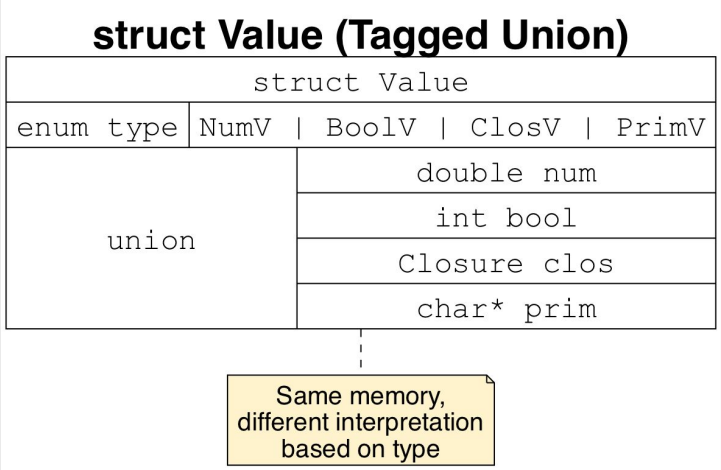
// C: if is a statement, returns nothing
if (x > 0) {
    y = 1;
}

// C: ternary is an expression, returns a value
y = (x > 0) ? 1 : 2;
```

C Types

Type Vanish After Compilation

- Static typing: checked at compile time, then erased
- Weak typing: 3.7 becomes 3 silently
- No runtime type info: we built tagged unions



Conclusion

Would I Take a C Job?

Verdict:

Yes for embedded systems, kernels, performance level work.

No for web applications.

C reveals what every other language hides.

Quote:

“C is quirky, flawed, and an enormous success.”
— Dennis Ritchie

Thank you